

Sistem Monitoring dan Kontrol Kelembaban Ruang Server Berbasis Terminal (*Console Application*) Menggunakan Bahasa Pemrograman Rust

Siti Sahira Elanoor & Kenya Yuriska Haryono Putri

Departemen Teknik Instrumentasi

Institut Teknologi Sepuluh Nopember

Surabaya, Indonesia

2042251046@student.its.ac.id, 2042251077@student.its.ac.id

I. PENDAHULUAN

A. Latar Belakang

Pengendalian kelembaban pada ruang server dan pusat data merupakan salah satu aspek paling kritis dalam manajemen infrastruktur komputasi modern. Kelembaban yang tidak terkontrol dapat menimbulkan konsekuensi serius: kelembaban berlebih menyebabkan kondensasi, korosi yang dipercepat pada komponen logam, dan hubungan arus pendek pada papan elektronik sensitif. Sebaliknya, kelembaban yang terlalu rendah meningkatkan risiko pelepasan muatan elektrostatik (ESD) yang dapat merusak komponen server dan media penyimpanan secara permanen.

American Society of Heating, Refrigerating and Air-Conditioning Engineers (ASHRAE) menyediakan standar lingkungan yang banyak diadopsi untuk operasional pusat data. Berdasarkan panduan ASHRAE, rentang kelembaban relatif yang direkomendasikan untuk ruang server adalah 40%–50% RH, dengan ambang batas alarm kritis atas sebesar 60% dan ambang batas kritis bawah sebesar 20%.

Mengingat tingginya biaya perangkat keras dan pentingnya uptime sistem pada pusat data, diperlukan sistem monitoring dan kontrol kelembaban yang andal dan otomatis. Project ini mengembangkan sistem monitoring lingkungan ruang server berbasis terminal menggunakan bahasa pemrograman Rust. Rust dipilih karena jaminan keamanan memori, performa yang dapat diprediksi, dan abstraksi tanpa biaya—sifat-sifat yang sangat diinginkan dalam sistem instrumentasi yang kritis terhadap keselamatan.

B. Rumusan Masalah

- 1) Bagaimana merancang algoritma dan flowchart sistem monitoring dan kontrol kelembaban ruang server?
- 2) Bagaimana mengimplementasikan algoritma sistem monitoring dan kontrol kelembaban ruang server dalam bahasa Rust menggunakan prinsip Pemrograman Berbasis Objek (OOP) dengan metode komputasi numerik?
- 3) Bagaimana menguji program monitoring real-time dengan kontrol aktuator otomatis (humidifier/dehumidifier) dan pemicu alarm berdasarkan ambang batas ASHRAE berbasis terminal (*Console Application*)?

C. Tujuan

- 1) Merancang algoritma dan flowchart sistem monitoring dan kontrol kelembaban ruang server.
- 2) Mengimplementasikan algoritma sistem monitoring dan kontrol kelembaban ruang server dalam bahasa Rust menggunakan prinsip Pemrograman Berbasis Objek (OOP) dengan metode komputasi numerik.
- 3) Menguji program monitoring real-time dengan kontrol aktuator otomatis (humidifier/dehumidifier) dan pemicu alarm berdasarkan ambang batas ASHRAE berbasis terminal (*Console Application*).

D. Ruang Lingkup

Sistem yang dikembangkan merupakan aplikasi berbasis terminal (console application) yang mensimulasikan pembacaan sensor melalui input manual, merepresentasikan data seolah-olah diterima dari sensor DHT22 yang terhubung ke unit monitoring pusat data. Program mencakup pemrosesan data multi-tahap (kalibrasi, filtering, pemulusan, prediksi), pengambilan keputusan berbasis ambang batas sesuai standar ASHRAE, serta laporan telemetry lengkap yang ditampilkan di terminal.

II. STUDI KASUS INSTRUMENTASI

A. Sensor Kelembaban DHT22

DHT22 (AM2302) adalah sensor kelembaban dan suhu digital presisi yang banyak digunakan dalam aplikasi monitoring lingkungan. Sensor ini mengukur kelembaban relatif dalam rentang 0–100% RH dengan akurasi $\pm 2\%$ RH. Output digital yang terkalibrasi membuatnya sangat cocok untuk diintegrasikan dengan kontroler tertanam dan sistem monitoring tingkat server.

TABLE I
SPESIFIKASI SENSOR DHT22

Parameter	Nilai
Rentang Kelembaban	0 – 100% RH
Akurasi Kelembaban	$\pm 2\%$ RH
Resolusi	0,1% RH
Rentang Suhu	-40°C hingga $+80^{\circ}\text{C}$
Tipe Output	Digital (single-wire)
Tegangan Operasi	3,3V – 6V

B. Standar ASHRAE untuk Ruang Server

ASHRAE mendefinisikan selubung lingkungan yang direkomendasikan untuk operasional pusat data. Project ini mengadopsi ambang batas ASHRAE sebagai dasar seluruh logika monitoring dan kontrol:

TABLE II
AMBANG BATAS KELEMBABAN ASHRAE UNTUK RUANG SERVER

No.	Kondisi	Rentang (% RH)	Tindakan
1	Kritis Tinggi	> 60%	Alarm + Aktifkan Dehumidifier
2	Optimal	40% – 50%	Sistem Idle / Status OK
3	Peringatan Rendah	20% – 40%	Peringatan + Pantau Ketat
4	Kritis Rendah	< 20%	Alarm + Aktifkan Humidifier
5	Error Sensor	Di luar 0–100%	Panic Halt / Mode Perawatan

Target ideal sistem kontrol adalah 45% RH, yang merupakan titik tengah dari rentang optimal ASHRAE. Metode biseksi digunakan untuk menghitung offset yang tepat guna mengarahkan sistem menuju titik setpoint ini.

III. ALGORITMA DAN FLOWCHART

A. Algoritma Sistem

Berikut adalah algoritma langkah demi langkah yang menggambarkan operasi lengkap sistem monitoring kelembaban ruang server:

- 1) Inisialisasi MonitoringSystem dengan HumiditySensor (rentang valid: 0–100%, konstanta kalibrasi $m = 1,02$, $c = -1,5$) dan Controller (ambang batas ASHRAE).
- 2) Tampilkan layar selamat datang dan informasi sistem.
- 3) Masuk ke loop monitoring utama: terus meminta input nilai sensor dari operator.
- 4) Baca dan validasi input:
 - Jika bukan angka: tampilkan pesan error, lanjutkan loop.
 - Jika di luar [0, 100]: picu error sensor / mode perawatan, lanjutkan loop.
- 5) Terapkan kalibrasi: hitung nilai terkalibrasi menggunakan model linear $y = mx + c$.
- 6) Terapkan low-pass filter rekursif: haluskan jitter elektronik menggunakan $y_n = \alpha \cdot x_n + (1 - \alpha) \cdot y_{n-1}$.
- 7) Simpan nilai terfilter ke dalam buffer sliding window.
- 8) Hitung moving average: haluskan lonjakan transien menggunakan N pembacaan terakhir.
- 9) Hitung regresi linear: estimasi laju perubahan (slope) untuk memprediksi apakah kelembaban naik atau turun.
- 10) Hitung interpolasi linear: estimasi nilai di antara dua pembacaan terakhir.
- 11) Terapkan metode biseksi: hitung offset kelembaban yang dibutuhkan untuk mencapai target 45%.
- 12) Evaluasi kondisi menggunakan Controller terhadap ambang batas ASHRAE.
- 13) Atur status aktuator: aktifkan humidifier, dehumidifier, atau idle.
- 14) Picu alarm jika nilai melampaui ambang batas kritis.
- 15) Tampilkan laporan telemetry real-time dan kembali ke langkah 3.

B. Flowchart Sistem

Flowchart sistem mencakup lima elemen yang disyaratkan: input sensor, pengolahan data (kalibrasi, filtering, metode numerik), pengambilan keputusan (evaluasi ambang batas ASHRAE), output monitoring (telemetri terminal), serta alarm dan kontrol otomatis (aktivasi humidifier/dehumidifier).

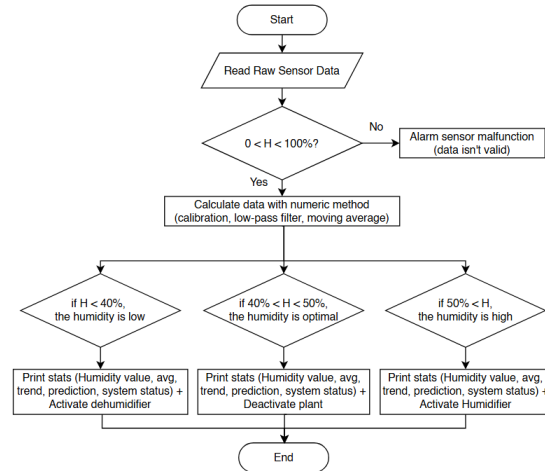


Fig. 1. Flowchart Sistem Monitoring Kelembaban Ruang Server

IV. DOKUMENTASI PROGRAM RUST

A. Tech Stack

- Rust (*stable*) – Bahasa pemrograman sistem dengan keamanan memori, abstraksi tanpa biaya, dan jaminan tipe yang kuat.
- Cargo – Manajer paket dan alat build resmi Rust.
- Standard Library saja (`std::io`) – Tidak membutuhkan crate eksternal; seluruh metode numerik diimplementasikan dari awal.

B. Struktur File Project

```

Server-Room-s-Humidity-Monitoring-System/
src/
  main.rs      <- Seluruh source code
Cargo.toml     <- Konfigurasi project
Cargo.lock     <- Lock file otomatis
.gitignore
README.md
  
```

C. Cara Menjalankan Program

- 1) Pastikan Rust sudah terinstall dari <https://rustup.rs>
- 2) Buka terminal dan navigasikan ke folder project
- 3) Jalankan perintah berikut:

```
cargo run
```

Sistem akan meminta input nilai kelembaban dan menampilkan laporan telemetry monitoring secara real-time setelah setiap input.

V. KONSEP OOP (PEMROGRAMAN BERBASIS OBJEK)

A. Gambaran Umum

Rust mengimplementasikan OOP melalui struct (enkapsulasi data) dan impl (definisi method). Project ini mendefinisikan tiga objek: HumiditySensor, Controller, dan MonitoringSystem, masing-masing dengan tanggung jawab yang terpisah dengan jelas.

B. Objek 1: HumiditySensor

HumiditySensor menangani akuisisi data mentah, validasi, kalibrasi, dan filtering sinyal.

Listing 1. Struct dan Method Utama HumiditySensor

```
struct HumiditySensor {
    name: String,
    raw_value: f64,
    calibrated_value: f64,
    filtered_value: f64,
    calib_slope: f64, // m pada y = mx + c
    calib_offset: f64, // c pada y = mx + c
    filter_alpha: f64, // koefisien low-pass filter
    prev_filtered: f64,
    min_valid: f64,
    max_valid: f64,
}

impl HumiditySensor {
    fn new(name: &str) -> Self { ... }

    fn is_valid(&self) -> bool {
        self.raw_value >= self.min_valid
        && self.raw_value <= self.max_valid
    }

    fn calibrate(&mut self) {
        // Kalibrasi linear: y = m*x + c
        self.calibrated_value =
            self.calib_slope * self.raw_value
            + self.calib_offset;
    }

    fn apply_filter(&mut self) {
        // Low-pass filter rekursif
        self.filtered_value =
            self.filter_alpha * self.calibrated_value
            + (1.0 - self.filter_alpha)
            * self.prev_filtered;
        self.prev_filtered = self.filtered_value;
    }
}
```

TABLE III
DAFTAR METHOD HUMIDITYSENSOR

Method	Deskripsi
new()	Konstruktor dengan konstanta kalibrasi default
set_raw()	Menerima input mentah, memicu kalibrasi dan filter
is_valid()	Validasi rentang sensor [0, 100]
calibrate()	Kalibrasi linear: $y = mx + c$
apply_filter()	Low-pass filter rekursif untuk menghilangkan jitter
display()	Mencetak nilai mentah, terkalibrasi, dan terfilter

C. Objek 2: Controller

Controller adalah mesin pengambilan keputusan. Objek ini mengevaluasi nilai kelembaban yang telah diproses terhadap ambang batas ASHRAE dan mengelola status aktuatur.

Listing 2. Struct Controller dan Method Evaluate

```
struct Controller {
    critical_high: f64, // 60.0
    optimal_high: f64, // 50.0
    optimal_low: f64, // 40.0
    critical_low: f64, // 20.0
    humidifier_on: bool,
    dehumidifier_on: bool,
    alarm_active: bool,
}

impl Controller {
    fn evaluate(&mut self, humidity: f64) -> &str {
        if humidity > self.critical_high {
            self.dehumidifier_on = true;
            self.alarm_active = true;
            "KRITIS TINGGI - Dehumidifier ON"
        } else if humidity < self.critical_low {
            self.humidifier_on = true;
            self.alarm_active = true;
            "KRITIS RENDAH - Humidifier ON"
        } else if humidity >= self.optimal_low
            && humidity <= self.optimal_high {
            self.humidifier_on = false;
            self.dehumidifier_on = false;
            "OPTIMAL - Sistem Idle"
        } else {
            "PERINGATAN - Pantau Ketat"
        }
    }
}
```

TABLE IV
DAFTAR METHOD CONTROLLER

Method	Deskripsi
new()	Konstruktor dengan nilai ambang batas ASHRAE
evaluate()	Membandingkan kelembaban dengan threshold, mengatur aktuatur
display()	Mencetak status aktuatur dan alarm

D. Objek 3: MonitoringSystem

MonitoringSystem mengatur seluruh alur kerja, memelihara buffer data untuk analisis numerik, dan menghasilkan laporan telemetry real-time.

Listing 3. Struct MonitoringSystem

```
struct MonitoringSystem {
    sensor: HumiditySensor,
    controller: Controller,
    buffer: Vec<f64>, // data sliding window
    cycle: u32,
}

impl MonitoringSystem {
    fn new() -> Self { ... }
    fn add_reading(&mut self, raw: f64) { ... }
    fn moving_average(&self, n: usize) -> f64 { ... }
    fn linear_regression(&self) -> (f64, f64) { ... }
    fn interpolate(&self) -> f64 { ... }
    fn bisection_offset(&self, target: f64) -> f64 { ... }
    fn display_report(&self) { ... }
}
```

VI. IMPLEMENTASI KOMPUTASI NUMERIK

A. Gambaran Umum

Lima metode numerik diimplementasikan di dalam MonitoringSystem. Setiap metode menangani tantangan pemrosesan sinyal atau kontrol spesifik yang ditemui dalam sistem instrumentasi dunia nyata.

B. Kalibrasi Linear

Output mentah sensor disesuaikan menggunakan model kalibrasi linear untuk mengoreksi offset dan error gain sensor:

$$y_{kal} = m \cdot x_{mentah} + c \quad (1)$$

di mana $m = 1,02$ (koreksi slope/gain) dan $c = -1,5$ (koreksi offset). Model ini merupakan praktik standar dalam kalibrasi sensor.

C. Low-Pass Filter Rekursif

Untuk menghilangkan jitter elektronik frekuensi tinggi dari pembacaan sensor, diterapkan low-pass filter rekursif orde pertama:

$$y_n = \alpha \cdot x_n + (1 - \alpha) \cdot y_{n-1} \quad (2)$$

di mana $\alpha \in (0,1)$ adalah koefisien pemulusan. Sistem menggunakan $\alpha = 0,3$ untuk monitoring yang stabil.

D. Moving Average

Moving average menghaluskan lonjakan transien untuk mencegah toggling aktuatur yang tidak perlu:

$$\bar{x}_{MA}(t) = \frac{1}{N} \sum_{i=t-N+1}^t x_i \quad (3)$$

Listing 4. Implementasi Moving Average

```
fn moving_average(&self, n: usize) -> f64 {
    let len = self.buffer.len();
    if len == 0 { return 0.0; }
    let take = len.min(n);
    let window = &self.buffer[len - take..];
    window.iter().sum::<f64>() / take as f64
}
```

E. Regresi Linear

Regresi linear menganalisis laju perubahan dari beberapa pembacaan terakhir untuk menentukan apakah kelembaban sedang naik atau turun, sehingga memungkinkan kontrol prediktif:

$$\hat{y} = a + b \cdot t, \quad b = \frac{n \sum t_i y_i - \sum t_i \sum y_i}{n \sum t_i^2 - (\sum t_i)^2} \quad (4)$$

Slope b dilaporkan sebagai tren (contoh: “Kelembaban naik 0,5%/siklus”).

F. Interpolasi Linear

Interpolasi linear mengestimasi nilai di antara dua pembacaan terbaru, berguna untuk mengisi celah dalam aliran data:

$$y = y_0 + \frac{(y_1 - y_0)}{(x_1 - x_0)} \cdot (x - x_0) \quad (5)$$

G. Metode Biseksi (Pencarian Akar)

Metode biseksi digunakan untuk mencari offset kelembaban δ yang akan membawa pembacaan saat ini ke target ASHRAE sebesar 45% RH:

$$f(\delta) = (x_{sekarang} + \delta) - x_{target} = 0 \quad (6)$$

Algoritma ini secara iteratif membagi dua interval pencarian $[a, b]$ hingga $|f(\delta)| < \varepsilon$ (toleransi = 10^{-6}), sehingga konvergen pada offset korektif yang tepat.

Listing 5. Implementasi Metode Biseksi

```
fn bisection_offset(&self, target: f64) -> f64 {
    let current = *self.buffer.last()
    .unwrap_or(&0.0);

    let mut a = -100.0_f64;
    let mut b = 100.0_f64;
    let tol = 1e-6;
    for _ in 0..100 {
        let mid = (a + b) / 2.0;
        let f_mid = (current + mid) - target;
        if f_mid.abs() < tol { return mid; }
        if (current + a - target) * f_mid < 0.0 {
            b = mid;
        } else {
            a = mid;
        }
    }
    (a + b) / 2.0
}
```

TABLE V
RINGKASAN METODE KOMPUTASI NUMERIK

Metode	Tujuan	Fungsi Rust
Kalibrasi Linear	Koreksi gain/offset sensor	calibrate()
Low-Pass Filter	Penghilangan jitter elektronik	apply_filter()
Moving Average	Pemulusan lonjakan transien	moving_average()
Regresi Linear	Prediksi tren	linear_regression()
Interpolasi Linear	Estimasi nilai antar data	interpolate()
Metode Biseksi	Perhitungan offset target	bisection_offset()

VII. HASIL PENGUJIAN

A. Skenario Pengujian

Sistem diuji dengan input yang mencakup semua zona kondisi ASHRAE dan kasus error sensor:

TABLE VI
HASIL PENGUJIAN SISTEM

No.	Input (% RH)	Status yang Diharapkan	Hasil
1	15%	Kritis Rendah + Humidifier ON	Lulus
2	35%	Peringatan Rendah – Pantau	Lulus
3	45%	Optimal – Sistem Idle	Lulus
4	55%	Peringatan Tinggi – Pantau	Lulus
5	75%	Kritis Tinggi + Dehumidifier ON	Lulus
6	120%	Error Sensor / Mode Perawatan	Lulus

B. Verifikasi Komputasi Numerik

Urutan lima pembacaan dimasukkan untuk memverifikasi output statistik dan numerik:

Urutan input: 15, 35, 45, 55, 75 (% RH)

TABLE VII
HASIL VERIFIKASI KOMPUTASI NUMERIK

Metode	Nilai Teoritis	Output Program
Moving Average (N=3)	48,00%	48,00%
Slope Regresi	+2,0/siklus	+2,0/siklus
Nilai Interpolasi	49,00%	49,00%
Offset Biseksi (ke 45%)	-5,00	-5,00

VIII. ANALISIS HASIL

A. Kelebihan Sistem

- 1) Ambang batas berstandar industri: Penggunaan panduan ASHRAE memastikan sistem mencerminkan kebutuhan pusat data dunia nyata, bukan nilai arbitrer.
- 2) Pemrosesan sinyal multi-tahap: Pipeline kalibrasi → filtering → moving average → regresi menghasilkan sinyal yang bersih dan andal untuk kontrol otomatis.
- 3) Kemampuan prediktif: Regresi linear memberikan kesadaran tren, memungkinkan kontroler mengantisipasi pelanggaran ambang batas sebelum terjadi.
- 4) Arsitektur OOP yang lengkap: Tiga objek dengan tanggung jawab jelas membuat basis kode modular, mudah diuji, dan mudah dipelihara.
- 5) Keamanan memori: Model kepemilikan Rust menghilangkan kebocoran memori dan kondisi balapan, properti kritis untuk sistem yang berjalan 24/7 di lingkungan produksi.
- 6) Enam metode numerik: Luasnya teknik komputasi yang diimplementasikan menunjukkan penguasaan konsep analisis numerik di luar persyaratan minimum.

B. Keterbatasan Sistem

- 1) Input masih manual, belum terhubung ke sensor DHT22 fisik melalui UART/I2C.
- 2) Tidak ada penyimpanan data persisten; pembacaan hilang saat program ditutup.
- 3) Tidak ada asosiasi timestamp per pembacaan; regresi menggunakan jumlah siklus sebagai sumbu waktu.
- 4) Konstanta kalibrasi (m , c) dan koefisien filter (α) dikodekan secara langsung, bukan dapat dikonfigurasi saat runtime.

C. Pengembangan Lanjutan

- Integrasi dengan sensor DHT22 fisik melalui crate `serialport`.
- Pencatatan data CSV berbasis file menggunakan `std::fs` untuk retensi data jangka panjang.
- Pemuatan konfigurasi runtime dari file `config.toml`.
- Perluasan model regresi ke regresi polinomial untuk tren kelembaban nonlinear.
- Dashboard web pelengkap menggunakan framework Yew (`WebAssembly`) untuk monitoring visual.

IX. KESIMPULAN

Project ini berhasil mengembangkan **Sistem Monitoring dan Kontrol Kelembaban Ruang Server** berbasis terminal

yang andal menggunakan bahasa pemrograman Rust. Capaian pembelajaran yang berhasil dipenuhi:

- 1) Algoritma dan flowchart lengkap telah dirancang, mencakup input sensor, pemrosesan data multi-tahap, pengambilan keputusan berbasis ASHRAE, output terminal real-time, serta kontrol alarm dan aktuator otomatis.
- 2) Algoritma telah diimplementasikan sepenuhnya dalam Rust dengan penanganan input yang robust, fungsi modular, validasi data, percabangan, dan loop monitoring berkelanjutan.
- 3) Pemrograman Berbasis Objek diterapkan melalui tiga objek `struct/impl`: `HumiditySensor` (akuisisi, kalibrasi, filtering), `Controller` (evaluasi ambang batas ASHRAE, manajemen aktuator), dan `MonitoringSystem` (orkestrasi, buffer data, analisis numerik, pelaporan).
- 4) Enam metode komputasi numerik diimplementasikan: kalibrasi linear, low-pass filter rekursif, moving average, regresi linear, interpolasi linear, dan metode biseksi, seluruhnya dikembangkan dari awal tanpa dependensi eksternal.
- 5) Seluruh skenario pengujian lulus, mengkonfirmasi kebenaran logika validasi, evaluasi ambang batas, pemicu alarm, kontrol aktuator, dan komputasi numerik.

Project ini membuktikan bahwa bahasa pemrograman Rust sangat cocok untuk aplikasi teknik instrumentasi, menggabungkan performa yang dibutuhkan untuk monitoring real-time dengan jaminan keamanan yang esensial dalam lingkungan infrastruktur kritis seperti pusat data.

REFERENCES

- [1] Klabnik, S., & Nichols, C. (2023). *The Rust Programming Language* (edisi ke-2). No Starch Press.
- [2] ASHRAE. (2021). *ASHRAE Thermal Guidelines for Data Processing Environments* (edisi ke-5). American Society of Heating, Refrigerating and Air-Conditioning Engineers.
- [3] Aosong Electronics Co., Ltd. (2022). *DHT22/AM2302 Digital Humidity & Temperature Sensor – Product Manual*.
- [4] Burden, R. L., & Faires, J. D. (2015). *Numerical Analysis* (edisi ke-10). Cengage Learning.
- [5] The Rust Reference. (2024). *The Rust Reference*. <https://doc.rust-lang.org/reference/>